

Proyecto de Sistema de Control y Medición de Tráfico

Entrega 2

Karol Torres Vides
Carrera de Ingeniería de Sistemas
Pontificia Universidad Javeriana
Bogotá D.C., Colombia
torres_kdayan@javeriana.edu.co

Mateo David Guerra Moreno
Carrera de Ingeniería de Sistemas
Pontificia Universidad Javeriana
Bogotá D.C., Colombia
mdavid.guerram@javeriana.edu.co

Resumen—Este documento presenta el diseño y el estado actual de implementación de un sistema distribuido para el control y la medición de tráfico urbano. La solución se organiza en tres nodos principales que cooperan mediante ZeroMQ para adquirir eventos de sensores, distribuirlos, analizarlos, almacenarlos de forma redundante y ejecutar acciones básicas sobre semáforos. El informe consolida los modelos del sistema, los diagramas de diseño, la estrategia de persistencia redundante, la configuración inicial de recursos, las reglas de operación, el protocolo de pruebas y la estrategia para obtener métricas de desempeño. El alcance descrito se limita a los componentes implementados en la versión actual del código fuente.

Index Terms—Sistemas distribuidos, ZeroMQ, replicación, persistencia redundante, control de tráfico, semáforos inteligentes.

I. INTRODUCCIÓN

El proyecto propone una plataforma distribuida para la gestión básica del tráfico urbano en una ciudad modelada como una cuadrícula de intersecciones. En cada intersección se simulan sensores capaces de producir eventos periódicos sobre volumen vehicular, velocidad promedio, conteo de vehículos y nivel de congestión. Estos eventos son transportados mediante ZeroMQ, procesados por un servicio de analítica y utilizados para tomar decisiones simples sobre el control de semáforos.

La solución se organiza en tres computadores con responsabilidades diferenciadas. PC_1 concentra la generación y distribución de eventos; PC_2 ejecuta la analítica, el control de semáforos y la base de datos réplica; y PC_3 ejecuta la base de datos principal. La versión actual implementa una estrategia de persistencia redundante, en la que cada evento procesado por analítica se envía tanto a la base de datos principal como a la réplica.

II. MODELOS DEL SISTEMA

II-A. Modelo arquitectónico

El sistema se distribuye en tres computadores:

- **PC_1:** aloja `sensores.py`, `broker_mq.py` y `config.py`. Su función es generar eventos simulados y redistribuirlos hacia el servicio de analítica.

- **PC_2:** aloja `analisis.py`, `control_semaforos.py`, `db_replica.py` y `config.py`. Este nodo realiza el procesamiento operativo, ejecuta comandos semafóricos y mantiene una copia redundante de los eventos.
- **PC_3:** aloja `db.py`, `db.json` y `config.py`. Este nodo representa la base de datos principal del sistema.

Desde el punto de vista arquitectónico, la solución se separa en cinco responsabilidades principales: adquisición de eventos, mensajería, procesamiento analítico, actuación semafórica y persistencia. Esta separación permite desacoplar la generación de datos, el análisis y el almacenamiento.

II-B. Modelo de interacción

La interacción del sistema se basa en dos patrones de comunicación de ZeroMQ:

- **PUB/SUB:** se emplea entre sensores, broker y analítica. Los sensores publican eventos con tópicos como `camara`, `espira` y `gps`; el broker recibe esos mensajes y los redistribuye al servicio de analítica.
- **PUSH/PULL:** se emplea desde analítica hacia la base de datos principal, la base de datos réplica y el control de semáforos. Este patrón permite enviar trabajo hacia componentes receptores sin requerir una respuesta explícita.

Cada evento generado por un sensor se publica hacia el broker. Luego, el broker lo reenvía a analítica. El servicio de analítica clasifica el evento como `NORMAL` o `CONGESTION`, lo almacena en la base principal y en la réplica, y cuando detecta congestión envía un comando al controlador de semáforos.

II-C. Modelo de fallos

En la versión actual se consideran fallos parciales asociados a pérdida de mensajes, indisponibilidad de componentes y diferencias entre los archivos de persistencia. El alcance implementado no incluye conmutación automática ni verificación activa de salud de nodos. Por tanto, el mecanismo de tolerancia se limita a la duplicación de eventos hacia dos destinos de almacenamiento.

II-C1. Clasificación de fallos considerados:

- **Fallo de un componente receptor:** un servicio como la base principal, la réplica o el control de semáforos puede no estar activo al momento de recibir mensajes.
- **Fallo de comunicación:** puede presentarse indisponibilidad temporal de sockets o pérdida de conectividad entre computadores.
- **Fallo de persistencia:** un evento puede no quedar almacenado en alguno de los archivos de salida si el componente correspondiente no se encuentra en ejecución.
- **Diferencia entre repositorios:** pueden existir diferencias entre la base principal y la réplica si alguno de los dos servicios no se ejecuta durante todo el experimento.

II-C2. Estrategia de mitigación implementada: La estrategia implementada es la **persistencia redundante**. Cada evento procesado por `analisis.py` se envía a dos componentes de almacenamiento: la base de datos principal y la base de datos réplica. Esta decisión permite conservar una copia secundaria de los eventos cuando ambos servicios están activos. En esta versión no se implementa detección automática de caída, redirección dinámica ni confirmación de escritura.

II-D. Modelo de seguridad

El modelo de seguridad se describe de acuerdo con el alcance real de la implementación. El sistema trabaja con datos simulados y no implementa autenticación de usuarios, cifrado de mensajes ni control formal de acceso. Aun así, la versión actual incorpora consideraciones básicas relacionadas con integridad, disponibilidad parcial y trazabilidad operativa.

- **Integridad básica:** los eventos se estructuran como mensajes JSON con campos definidos según el tipo de sensor.
- **Disponibilidad parcial:** los eventos procesados se envían tanto al repositorio principal como al repositorio réplica.
- **Trazabilidad operativa:** los servicios imprimen en consola los eventos generados, recibidos, procesados, almacenados y los comandos enviados al controlador de semáforos.
- **Limitaciones:** no se implementan credenciales, roles, cifrado, firmas digitales ni auditoría formal de acciones.

II-E. Modelo de McCumber aplicado

El modelo de McCumber se usa para relacionar los objetivos de seguridad, los estados de la información y las salvaguardas existentes en el sistema. En este proyecto, los activos principales son los eventos de tráfico, los comandos semafóricos y los registros persistidos en archivos locales.

La aplicación del modelo muestra que el énfasis principal de la versión actual está en la integridad estructural de los mensajes y en la disponibilidad parcial mediante persistencia redundante. Los controles de confidencialidad se reconocen como limitados, dado que el prototipo no implementa cifrado ni autenticación.

III. DISEÑO DEL SISTEMA

III-A. Diagrama de despliegue

La Figura 2 presenta la distribución física de los artefactos implementados. El diagrama muestra qué archivos se ejecutan

en cada computador y los patrones de comunicación usados entre ellos.

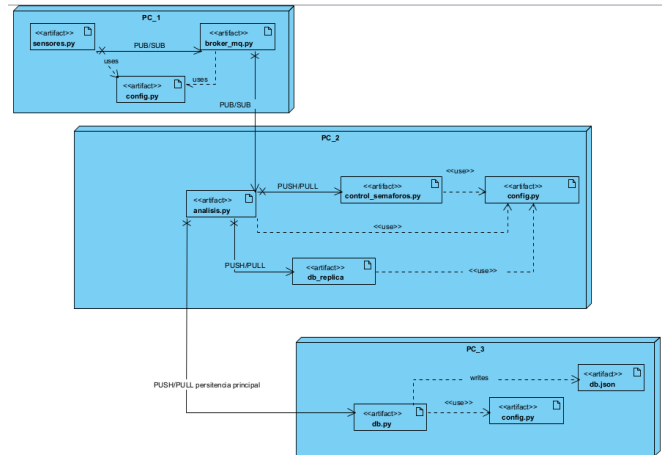


Figura 2: Diagrama de despliegue del sistema distribuido de tráfico.

Descripción del diagrama. En PC_1 se ubican `sensores.py`, `broker_mq.py` y `config.py`. En PC_2 se ubican `analisis.py`, `control_semaforos.py`, `db_replica.py` y `config.py`. En PC_3 se ubican `db.py`, `db.json` y `config.py`. Las flechas muestran la comunicación PUB/SUB entre sensores, broker y analítica, así como la comunicación PUSH/PULL desde analítica hacia la base principal, la réplica y el control de semáforos.

III-B. Diagrama de componentes

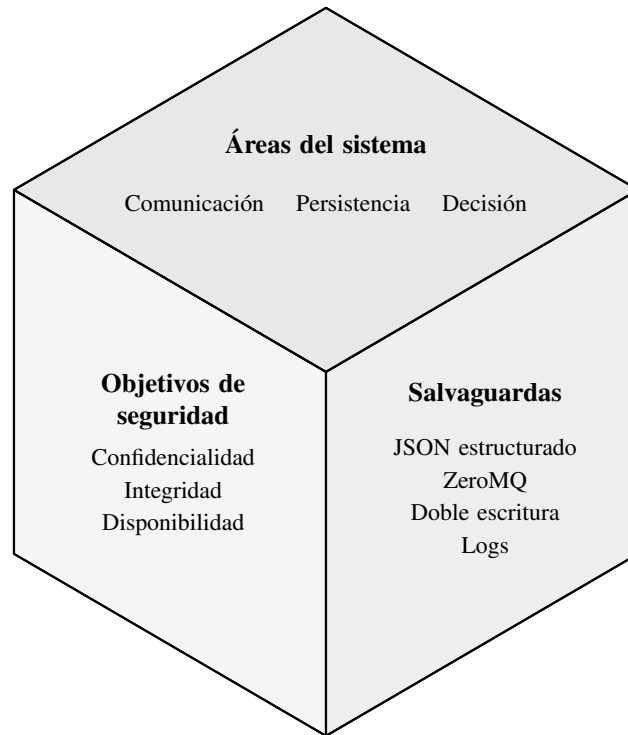
La Figura 3 muestra la descomposición lógica del sistema por responsabilidades funcionales. A diferencia del diagrama de despliegue, este modelo no agrupa los elementos por computador, sino por función dentro del sistema.

Descripción del diagrama. El sistema se divide en adquisición de eventos, mensajería, procesamiento, actuación y persistencia. El componente de sensores genera eventos simulados; el broker ZMQ los redistribuye; el servicio de analítica los procesa y aplica reglas de tráfico; el control de semáforos ejecuta acciones cuando se detecta congestión; y los repositorios principal y réplica almacenan los eventos procesados. La réplica se representa como un segundo repositorio de persistencia que recibe los mismos eventos enviados por analítica.

III-C. Diagrama de clases

La Figura 4 detalla la estructura lógica del software. El diagrama no representa necesariamente un archivo por clase, sino las responsabilidades internas derivadas del código implementado.

Descripción del diagrama. La jerarquía de sensores se organiza a partir de una clase base `Sensor`, especializada en `CamaraSensor`, `EspiraSensor` y `GPSSensor`. La clase `EventoTráfico` representa la unidad de información intercambiada. `BrokerMQ` recibe y redistribuye eventos, mientras



Aplicación al proyecto: los activos protegidos del sistema son los eventos de tráfico, los comandos semafóricos y los registros persistidos. En la versión implementada, la **integridad** se apoya en mensajes JSON estructurados y en el procesamiento de reglas de analítica; la **disponibilidad** se refuerza mediante el envío de eventos tanto al repositorio principal como al repositorio réplica; y la **confidencialidad** es limitada, ya que el prototipo no implementa autenticación, cifrado, roles de usuario ni auditoría formal.

Figura 1: Cubo de McCumber aplicado al sistema de control y medición de tráfico.

Tabla I: Aplicación del modelo de McCumber al sistema implementado

Parte del sistema	Confidencialidad	Integridad	Disponibilidad
El prototipo opera en un entorno controlado de pruebas, sin autenticación ni cifrado formal de mensajes.	Los eventos se intercambian como mensajes JSON con campos definidos por tipo de sensor.	La comunicación se desacopla mediante ZeroMQ usando los patrones PUB/SUB y PUSH/PULL.	Comunicación entre servicios Los archivos locales de eventos se manejan dentro del entorno de ejecución del proyecto.
Los registros se almacenan como eventos JSON procesados por el servicio de analítica.	Cada evento procesado se envía tanto al repositorio principal como al repositorio réplica.	Persistencia de eventos Las decisiones del sistema no exponen datos personales, ya que los eventos son simulados.	El servicio de analítica clasifica eventos como NORMAL o CONGESTION según reglas definidas.
El sistema mantiene una copia redundante de los eventos y ejecuta comandos semafóricos cuando detecta congestión.	Procesamiento y decisión		

que `ServicioAnalitica` evalúa reglas y genera acciones hacia `ControlSemaforos`. La persistencia se abstrae mediante una única clase `RepositorioEventos`, de la cual se derivan dos instancias operativas: `bdPrincipal` y `bdReplica`. Esta representación evita duplicar clases para la base principal y la réplica, ya que ambas cumplen la misma responsabilidad lógica de recibir y guardar eventos.

III-D. Diagramas de secuencia

Para evidenciar el comportamiento dinámico del sistema se presentan dos secuencias: una de procesamiento normal y otra de persistencia redundante.

III-D1. Secuencia de procesamiento normal: Descripción del diagrama. Un sensor publica un evento al broker, el

broker lo reenvía al servicio de analítica y analítica evalúa el evento según su tópico. Luego, el evento procesado se envía al repositorio principal y al repositorio réplica. Si el estado calculado corresponde a `CONGESTION`, analítica envía un comando `EXTENDER_VERDE` al controlador de semáforos. Si el estado es `NORMAL`, no se emite acción semafórica.

III-D2. Secuencia de persistencia en caso de fallo de la DB Principal: Descripción del diagrama. La Figura 6 muestra cómo el servicio de analítica envía el mismo evento procesado hacia dos destinos de almacenamiento: el repositorio principal y el repositorio réplica. Ambos repositorios reciben los mensajes mediante el patrón `PUSH/PULL` y los almacenan en archivos locales. En esta versión no se modela

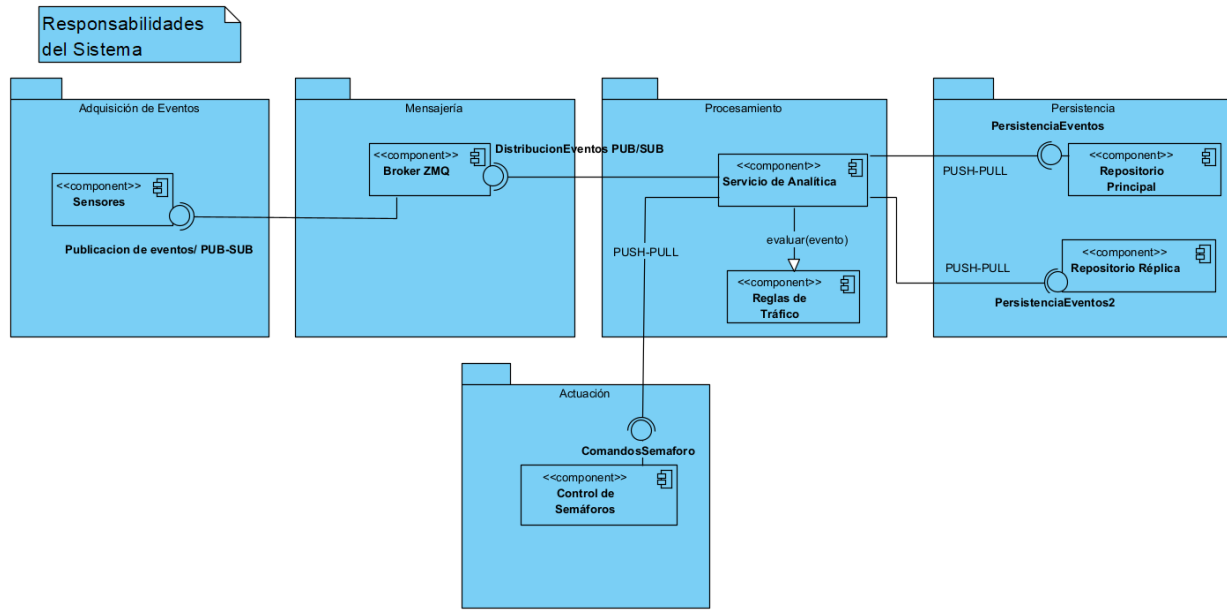


Figura 3: Diagrama de componentes del sistema.

una respuesta de confirmación desde las bases de datos, ya que los servicios de persistencia reciben los mensajes y los guardan sin enviar una respuesta explícita al servicio de analítica.

IV. INICIALIZACIÓN DEL SISTEMA

La configuración inicial se centraliza en archivos `config.py`, los cuales definen direcciones IP, puertos, intersecciones y parámetros globales de simulación. Esta decisión facilita ejecutar el sistema en varios computadores o ajustar sus direcciones según el entorno de prueba.

IV-A. Parámetros base

- Direcciones de red de los computadores: `PC1_IP`, `PC2_IP` y `PC3_IP`.
- Direcciones de sockets ZeroMQ para cada tipo de comunicación.
- Lista de intersecciones simuladas, como `INT_A1`, `INT_A2`, `INT_B1` e `INT_B2`.
- Intervalo entre mediciones de sensores.
- Tiempo base de fase verde para los semáforos.

IV-B. Recursos iniciales del sistema

Para la primera entrega se asume una ciudad mínima con una cuadrícula reducida. La definición inicial de recursos incluye:

- sensores simulados de cámara, espira inductiva y GPS;
- intersecciones definidas en la configuración;
- servicio de broker para redistribución de eventos;
- servicio de analítica para clasificación de eventos;
- controlador de semáforos para ejecutar comandos;
- repositorio principal y repositorio réplica para persistencia redundante.

V. REGLAS DEL SISTEMA

Las reglas del sistema traducen las mediciones de sensores en estados de operación. En la versión actual se utilizan reglas simples dentro de `analysis.py`.

V-A. Eventos de cámara

Un evento de cámara se clasifica como congestión cuando el volumen vehicular es alto o cuando la velocidad promedio es baja:

$$[\text{volumen} > 8 \quad \vee \quad \text{velocidad}_{\text{promedio}} < 15]$$

Si no se cumple ninguna de estas condiciones, el evento se clasifica como `NORMAL`.

V-B. Eventos de espira inductiva

Un evento de espira se clasifica como congestión cuando el número de vehículos contados supera el umbral definido:

$$[\text{vehiculos}_{\text{contados}} > 15]$$

En caso contrario, el estado asignado es `NORMAL`.

V-C. Eventos GPS

Un evento GPS se clasifica como congestión cuando la velocidad promedio es menor al umbral mínimo:

$$[\text{velocidad}_{\text{promedio}} < 10]$$

Si la condición no se cumple, el evento se clasifica como `NORMAL`.

V-D. Acción semafórica

Cuando analítica clasifica un evento como `CONGESTION`, se envía al controlador de semáforos un comando con la acción `EXTENDER_VERDE`. Esta acción actualiza el estado local de la intersección afectada e imprime la decisión en consola.

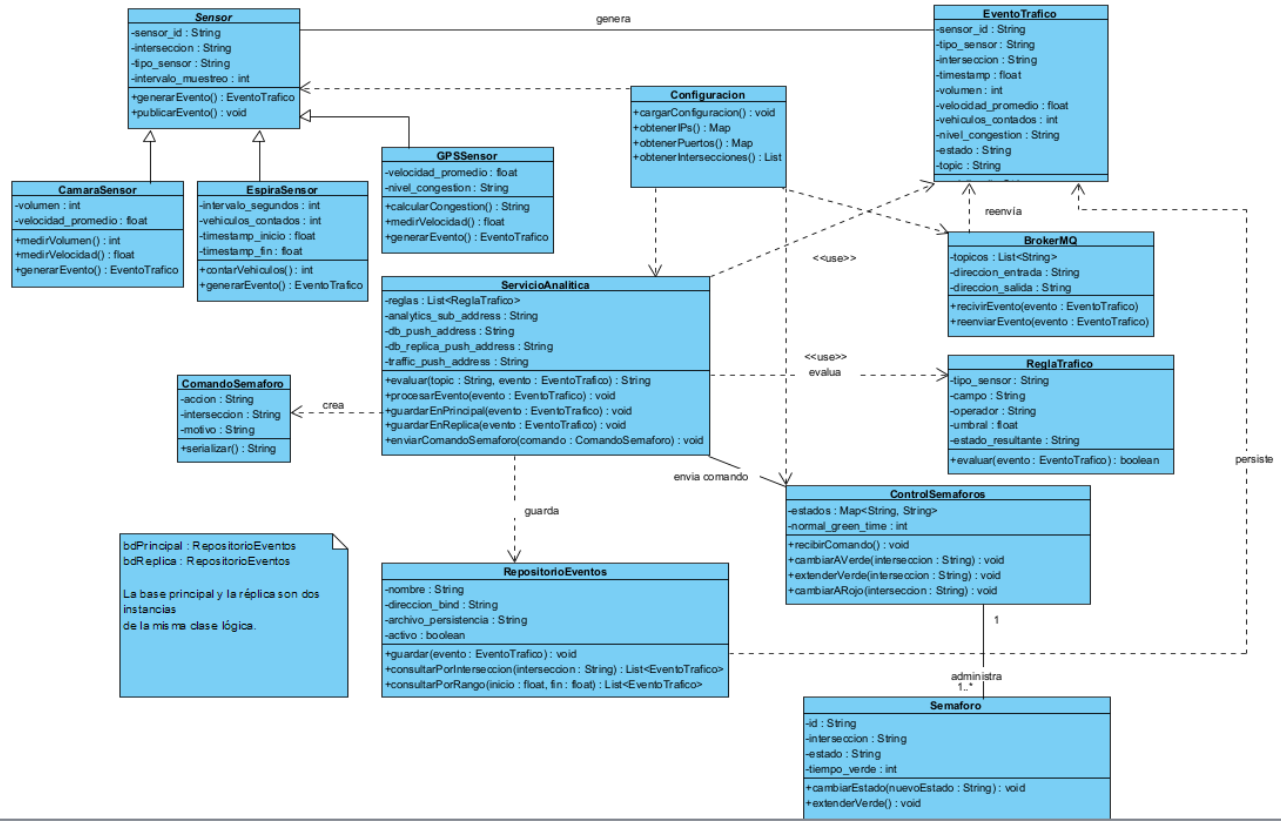


Figura 4: Diagrama de clases del sistema.

VI. OBSERVABILIDAD Y EJECUCIÓN DE PRUEBAS

La versión actual no implementa un módulo de consulta de usuario. La observación del comportamiento del sistema se realiza mediante las salidas por consola y los archivos de persistencia.

VI-A. Evidencias observables

- **Sensores:** imprimen los eventos generados y el tópico asociado.
- **Broker:** imprime los mensajes recibidos y redistribuidos.
- **Análítica:** imprime los eventos procesados y el estado asignado.
- **Control de semáforos:** imprime los comandos recibidos y el estado actualizado.
- **Repositorios:** imprimen los eventos almacenados y los escriben en archivos locales.

VII. DESCRIPCIÓN DEL CÓDIGO FUENTE IMPLEMENTADO

La primera entrega cuenta con servicios distribuidos para PC_1, PC_2 y PC_3. En el estado actual del proyecto, los archivos implementados cumplen las siguientes responsabilidades:

VII-A. config.py

Centraliza direcciones IP, puertos, nombres de intersecciones y parámetros globales de simulación. Su objetivo es

separar la configuración de red de la lógica principal de los servicios.

VII-B. sensores.py

Simula la generación de eventos de cámara, espira inductiva y GPS. Cada evento incluye información como identificador del sensor, tipo de sensor, intersección, mediciones y marca de tiempo. Los eventos se publican hacia el broker mediante el patrón PUB/SUB.

VII-C. brokermq.py

Funciona como intermediario de mensajería. Se suscribe a los tópicos *camara*, *espira* y *gps*, recibe eventos de sensores y los redistribuye al servicio de analítica.

VII-D. analisis.py

Es el núcleo de procesamiento del sistema. Recibe eventos desde el broker, evalúa reglas simples según el tipo de sensor, asigna el estado *NORMAL* o *CONGESTION*, envía el evento procesado a la base principal y a la réplica, y genera comandos hacia el control de semáforos cuando detecta congestión.

VII-E. controlsemaforos.py

Recibe comandos generados por analítica y actualiza el estado local de los semáforos. Cuando recibe la acción *EXTENDER_VERDE*, cambia el estado de la intersección a *VERDE_EXTENDIDO* e imprime la acción realizada.

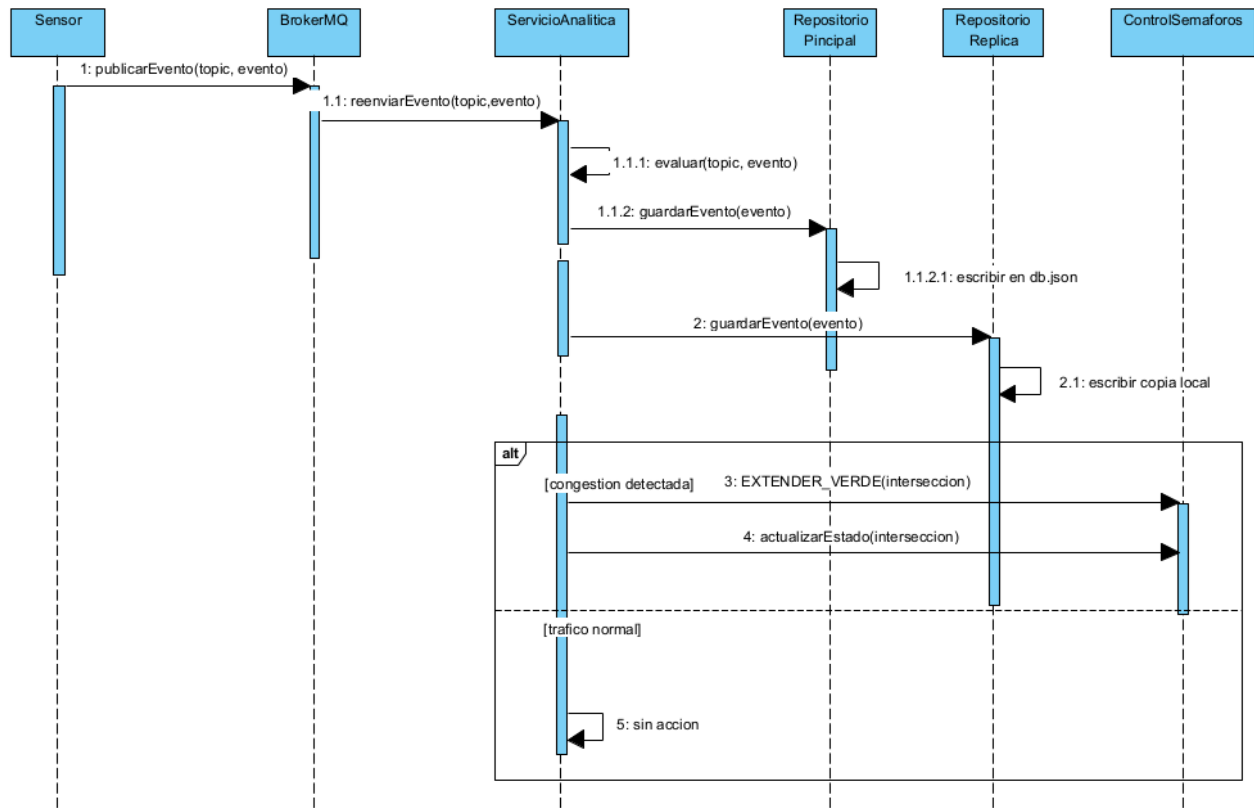


Figura 5: Diagrama de secuencia para el procesamiento normal de un evento.

VII-F. *db.py*

Representa el repositorio principal del sistema. Recibe eventos mediante un socket PULL y los almacena en un archivo JSON local.

VII-G. *db_replica.py*

Representa el repositorio réplica del sistema. Recibe los mismos eventos procesados por analítica y los almacena como copia redundante.

VII-H. *db.json*

Archivo de persistencia utilizado para registrar eventos almacenados durante la ejecución del sistema.

VIII. PROTOCOLO DE PRUEBAS

El protocolo de pruebas se plantea de forma incremental, iniciando con validaciones funcionales, continuando con pruebas de integración y terminando con pruebas de desempeño.

VIII-A. Pruebas funcionales

- **Caso 1. Generación de eventos:** verificar que los sensores produzcan mensajes con estructura válida y frecuencia definida.
- **Caso 2. Reenvío por broker:** verificar que los eventos publicados por sensores sean recibidos y redistribuidos por el broker.

- **Caso 3. Evaluación de reglas:** validar que analítica clasifique eventos como NORMAL o CONGESTION.
- **Caso 4. Control de semáforos:** verificar que los comandos de congestión sean recibidos por control_semaforos.py.
- **Caso 5. Persistencia principal:** verificar que *db.py* reciba y almacene eventos.
- **Caso 6. Persistencia réplica:** verificar que *db_replica.py* reciba y almacene una copia de los eventos.

VIII-B. Pruebas de integración

- **Caso 7. Flujo extremo a extremo:** ejecutar simultáneamente los servicios de PC_1, PC_2 y PC_3, y verificar que un evento recorra sensores, broker, analítica, persistencia y control semafórico.
- **Caso 8. Persistencia redundante:** comprobar que un evento procesado por analítica sea enviado tanto al repositorio principal como al repositorio réplica.

VIII-C. Pruebas de desempeño

Las pruebas de desempeño se concentran en dos variables dependientes:

1. cantidad de eventos almacenados durante una ventana de dos minutos;
2. tiempo transcurrido entre la clasificación de congestión en analítica y la ejecución del comando en el controlador de semáforos.

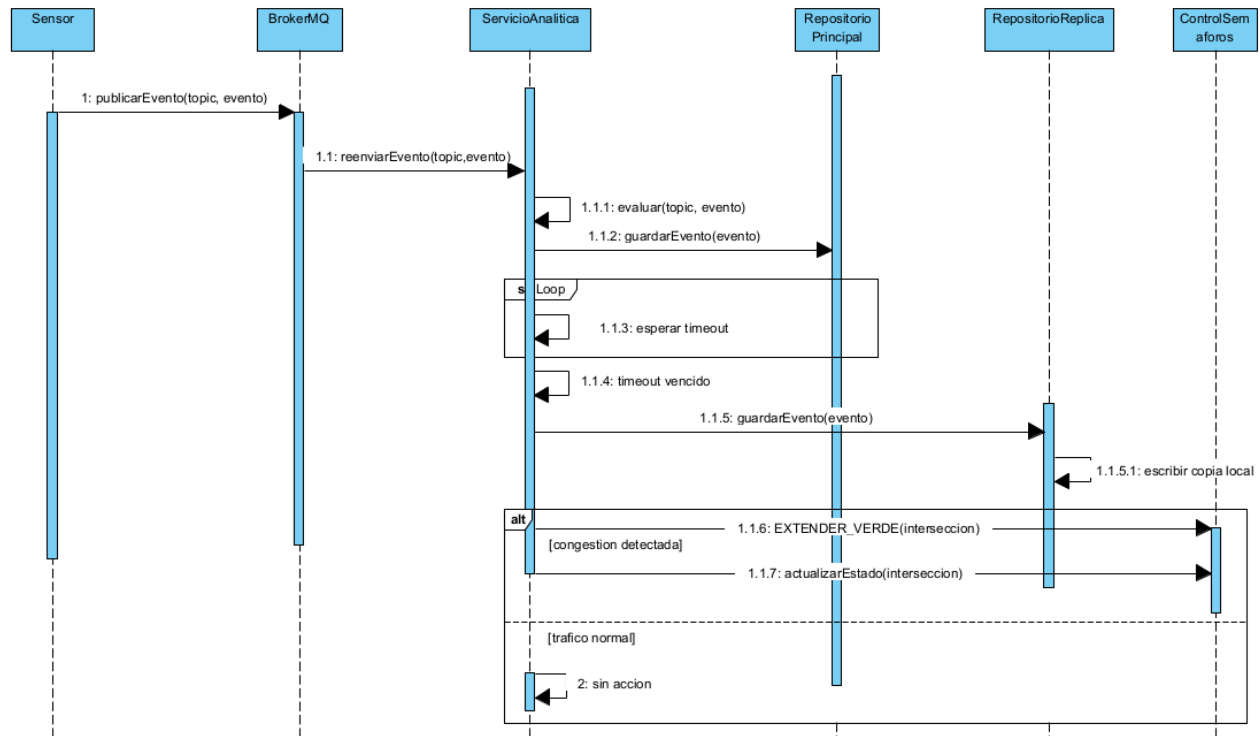


Figura 6: Diagrama de secuencia de persistencia en caso de fallo de la DB Principal.

Se evaluarán, como mínimo, los siguientes escenarios:

- **Escenario 1 (carga baja):** eventos generados con un intervalo amplio entre mediciones.
- **Escenario 2 (carga media):** eventos generados con menor intervalo entre mediciones.

IX. OBTENCIÓN DE MÉTRICAS DE DESEMPEÑO

La estrategia de medición se basa en marcas de tiempo, conteo de registros y observación de logs en consola.

IX-A. Métrica 1: eventos almacenados en dos minutos

Se contará el número de eventos persistidos en los archivos de almacenamiento durante una ventana fija de dos minutos. Este valor puede obtenerse contando las líneas válidas en los archivos generados por la base principal y la réplica.

IX-B. Métrica 2: tiempo de reacción analítica-semáforo

Se registrará un tiempo aproximado desde que analítica detecta congestión hasta que el controlador de semáforos imprime la ejecución del comando. Esta medición permite estimar la latencia observable del flujo de decisión y actuación.

IX-C. Herramientas de medición

- salidas por consola de los servicios;
- archivos de persistencia de la base principal y la réplica;
- conteo de registros almacenados;
- comparación de eventos procesados frente a comandos semafóricos ejecutados.

IX-D. Criterios de evaluación

Se considerará que el sistema presenta un comportamiento aceptable si:

- los eventos generados por sensores llegan al servicio de analítica;
- analítica clasifica correctamente los eventos según las reglas definidas;
- los eventos procesados se envían tanto al repositorio principal como al repositorio réplica;
- los comandos de congestión llegan al controlador de semáforos;
- el sistema mantiene tiempos de respuesta consistentes bajo los escenarios de carga definidos.

X. CONCLUSIONES

La versión actual del sistema implementa un flujo distribuido básico para adquisición, mensajería, análisis, persistencia redundante y control semafórico. El diseño corregido diferencia el despliegue físico de los componentes lógicos, evita representar clases duplicadas para la base principal y la réplica, y ajusta los diagramas de secuencia al comportamiento real del código. En particular, la persistencia se modela como el envío del mismo evento hacia dos repositorios, sin asumir confirmaciones, consultas de usuario ni conmutación automática no implementada.